AI-Assisted Software Development Policy

Status: Draft v0.1 — an example, not a standard
Applies to: Any software work done with help from AI coding tools — chat agents, autocomplete, plan or spec generators, review bots.
Who it's for: Engineers, reviewers, release owners, and the person who signs off on each change.

> **An example, not a prescription.** This document shows one way a team might govern AI-assisted development. It is a starting point to adapt or replace, not a standard to adopt as written, and it carries no endorsement from any standards body. AI-assisted development is new, and teams are still working out what good practice looks like. It is shared here so others can borrow from it, argue with it, and publish better versions. See §10 for other approaches worth reading.

---

1. Why this exists

AI tools change who does the work, but not who is responsible for it. A person still owns every change, even one an AI mostly wrote.

This policy keeps AI-assisted work under control in three plain ways:

- Every change can be traced back to a clear reason for making it.
- Every change is checked by someone other than whoever (or whatever) made it.
- Every change leaves a record you can look at later.

This isn't a manual for any one tool. Where a rule already has a working implementation, we point to `plan-tool` as an example. The rule is what matters; any tool that meets it is fine.

2. The main idea

> **AI doesn't give the same answer twice. Our job is to put steady, repeatable checks around it and keep a record of what happened.**

That comes down to three rules we don't bend:

1. **Always surface problems.** Findings, risks, and "there's a better way to do this" must always show up. Nobody gets to quietly turn them off.
2. **Limit what AI can change on its own.** AI can change things only within set limits, only when it's safe, and never for big or hard-to-undo work.
3. **Keep a durable record.** Every decision and check leaves an artifact that outlives the chat window.

3. Terms we use

3.1 Who's who

- **Assistant** — any AI tool that writes plans, code, or reviews. It never gives final sign-off.
- **Author** — the person responsible for a change, whether or not AI helped.
- **Checker** — a person or a separate automated pass that reviews the author's work. It must not be the same pass that made it.
- **Approver** — the named person who signs off on a merge or release. Always a person.

3.2 How much AI can do on its own

Each *kind* of change gets one of three levels. The level decides what AI may do without a person stepping in.

Level	AI can…	Only if…	Example in plan-tool
L0 — Suggest	Point things out, recommend, comment	Always allowed	granularity advisor, scope findings
L1 — Change with a yes	Edit code or plans	A person says yes *and* it's safe to do	`escalate_to_split` (safe + allowed + confirmed)
L2 — Person-led	Draft and suggest, with a person steering	A person is in the loop and decides each step	reshaping a plan's structure

Big or hard-to-undo changes — database migrations, public API changes, security settings, adding new dependencies — start at **L2** unless an Approver decides otherwise.

What L2 means in practice. L2 is about *who is steering*, not who types the words. Creating a plan is usually a back-and-forth conversation with an AI assistant: the AI drafts most of the text, but a person is reading, redirecting, and accepting or rejecting at every turn. That's fine, and it's the normal way to work — the person is leading even though the AI did the writing. What L2 rules out is the opposite: an AI reshaping something significant *on its own, with nobody watching* (a batch or agent run that restructures a plan unattended). The line is **attended vs. unattended**, not **AI-wrote vs. human-wrote**.

One caveat: "a person was in the chat" is not the same as "a person reviewed the result." An L2 change made through conversation still goes into version control and still gets the independent check at review (§5 Review). The conversation is how the work gets authored — not a replacement for checking it.

4. The five things we care about

1. **Traceability** — every change links back to a clear, written reason.
2. **Human sign-off** — a named person approves every merge; AI never does.
3. **Independent checking** — work is reviewed by someone other than who made it.
4. **Provenance** — we label AI-made changes, note which model and version made them, and scan for secrets, license issues, and risky dependencies.
5. **Tuning over time** — we log how AI decisions go and adjust the rules based on what we see.

5. What to do at each stage

Each stage below lists the **control**, how much AI can do on its own (**level**), and the **record** it leaves. "Ref" points to the plan-tool feature that does it, where one exists.

Idea / intake

- **Traceability:** Save the rough idea as a file in version control. *L0.* **Record:** seed file. *Ref: `plan-seeds/`, `create --from-existing`.*
- **Provenance:** Note that AI helped and which model version. *L0.* **Record:** origin note.

Planning

- **Traceability:** Turn the idea into a structured plan — ordered steps with details — that serves as the source of truth. The prose is for people; the structure is what counts. *L1.* **Record:** plan file. *Ref: `create → detail`.*
- **Human sign-off:** Splitting or merging steps comes up as a suggestion; it's only applied with a yes and a safety check. *L1/L2.* **Record:** the decision and its outcome. *Ref: `escalate_to_split`, granularity advisor.*
- **Checking:** Audit the plan until no new problems come up. *L0.* **Record:** audit reports. *Ref: `completeness-audit`.*

Building

- **Traceability:** Every change ties to a plan step; anything outside the plan's scope gets flagged. *L1.* **Record:** run log + scope flags. *Ref: `run`, `scope_guard.py`.*
- **Human sign-off:** AI can make step-sized changes (L1); reshaping the work is person-only (L2). **Record:** change decisions.
- **Provenance:** Label AI-made changes and note the model version. *L0.* **Record:** commit note.

Review

- **Checking:** A separate review pass — not the one that wrote the code — looks for bugs and gaps. *L0.* **Record:** review findings. *Ref: `review`, `/code-review`.*
- **Human sign-off:** Findings always show up. Ignoring one takes a recorded human decision. **Record:** how each finding was handled.
- **Provenance / risk:** Scan the change for secrets, license problems, and risky dependencies. *L0, blocks on protected branches.* **Record:** scan reports. *(Comes from other tools, not plan-tool.)*

Merge / CI

- **Checking:** Findings are advisory while developing, but **block the merge** on protected branches. **Record:** CI result. *Ref: exit-code rules + `--fail-on-findings`.*
- **Human sign-off:** A named person must approve the merge. AI never approves. **Record:** approval.

Release

- **Provenance:** The release notes which AI helped, which model versions, and what findings were left open and why. *L0.* **Record:** release manifest.
- **Tuning:** Capture how often gates fired and were overridden this cycle. *L0.* **Record:** decision log. *Ref: `telemetry.py`.*

Ongoing

- **Tuning:** Look at the pattern of AI decisions, overrides, and flags over time, and adjust the rules from what you see — not from one bad day. *L0.* **Record:** review notes. *Ref: decision log;* `convergence <name>` *reports advisory loop trends for a plan, not governance effectiveness.*
- **Drift:** Before execution, re-check live plans against current code with code as the source of truth; after execution, check the implementation against the plan with the plan as the source of truth. *L1.* **Record:** reconcile reports. *Ref:* `reconcile-plan --refresh` *(pre-execution),* `reconcile-code --all` *(post-execution).*

6. How we enforce it

- **Protected branches** run the review and scan checks with findings set to **block** the merge. Finishing a step is not the same as passing it.
- **No merge without:** a linked reason, an independent check, and a named human approver.
- **Going past the limits** — like AI making an L2 change on its own — is a policy problem. We log it and review it.
- **Ignoring a finding** takes a recorded human decision with a reason. Quietly dropping one is not allowed.

7. Who's responsible for what

Job	**Owner**
Making a change (with AI help)	Author (a person)
Independent check	Checker (a person or a separate automated pass)
Approving the merge or release	Approver (a named person)
Setting the level for each kind of change	Approver / repo owner
Reviewing how the rules are working	Eng lead / governance owner

AI can do the *work* of writing, checking, and suggesting. It never carries the *responsibility* of approving.

8. How often we review the rules

- **Each release:** count how often gates fired and were overridden.
- **Monthly (or every few releases):** look at the AI decision log, find where checks fired needlessly or missed, and adjust.
- **Quarterly:** revisit the levels and the scanning tools as AI tools get more capable.

9. What this doesn't cover

plan-tool covers planning, scope, review orchestration, and the decision log well. It does **not** on its own give you:

- **Labeling AI-made changes** at the commit level.
- **Scanning for secrets, license problems, and risky dependencies.**
- **Tracking which model version made a change** for later investigation.

You'll need other tools for these — secret scanners, dependency scanners, SBOM tools, and a commit-labeling convention. They're marked as such in §5.

10. Other approaches worth reading

This policy is one example among several possible ones, and `plan-tool` is one way to meet it. Two pointers for comparison:

- **AI-DLC (AI-Driven Development Life Cycle)** — an open-source methodology from AWS Labs, published at awslabs/aidlc-workflows under the MIT-0 license. It supplies workflow rules that steer an AI coding assistant through structured phases with human approval gates, across several assistants and IDEs. It is a credible alternative to the approach here, and the two can inform each other. This policy has not been checked against it line by line.
- **NIST SSDF (SP 800-218)** — the secure-development framework that Appendix A maps this policy against.

If you maintain another approach, or find one of these rules wrong, please say so by opening an issue or pull request.

Appendix A — Lining up with NIST SSDF (SP 800-218)

A light crosswalk if you need one:

SSDF group	Covered in	Pillar
PO — set up roles and levels	§3, §7	Human sign-off
PS — protect the software (provenance)	§5 Release, §9	Provenance
PW — build it well (planning, review)	§5 Planning–Review	Traceability, Checking
RV — respond to problems (findings, tuning)	§6, §8	Checking, Tuning

Expand this per-practice if you need a full audit trail.

Appendix B — Adapting this

- **Publishing it for others:** move the plan-tool mentions out of the main text into a separate "example tools" appendix so the rules stand on their own.
- **Targeting SOC 2 or ISO 27001:** add a section on where each record is kept and for how long, plus an owner for each control.
- **Letting AI do more on its own:** if you widen L1, review more often (§8) and make the review pass fully independent — a different model or agent than the one that wrote the code.